

# websocket/\_core.py

The \_core.py file

```
class websocket._core.WebSocket(get_mask_key: Callable | None = None, sockopt: list | None = None, sslopt: dict | None = None, fire_cont_frame: bool = False, enable_multithread: bool = True, skip_utf8_validation: bool = False, dispatcher: DispatcherBase | WrappedDispatcher | None = None, **_: Any) [source]
```

Low level WebSocket interface.

This class is based on the WebSocket protocol [draft-hixie-thewebsocketprotocol-76](#)

We can connect to the websocket server and send/receive data. The following example is an echo client.

```
>>> import websocket
>>> ws = websocket.WebSocket()
>>> ws.connect("ws://websockets.chilkat.io/wsChilkatEcho.ashx")
>>> ws.send("Hello, Server")
19
>>> ws.recv()
'Hello, Server'
>>> ws.close()
```

## Parameters:

- **get\_mask\_key** (*func*) – A callable function to get new mask keys, see the `WebSocket.set_mask_key`'s docstring for more information.
- **sockopt** (*tuple*) – Values for `socket.setsockopt`. `sockopt` must be tuple and each element is argument of `sock.setsockopt`.
- **sslopt** (*dict*) – Optional dict object for ssl socket options. See FAQ for details.
- **fire\_cont\_frame** (*bool*) – Fire `recv` event for each `cont` frame. Default is `False`.
- **enable\_multithread** (*bool*) – If set to `True`, lock `send` method.
- **skip\_utf8\_validation** (*bool*) – Skip utf8 validation.

```
__init__(get_mask_key: Callable | None = None, sockopt: list | None = None, sslopt: dict | None = None, fire_cont_frame: bool = False, enable_multithread: bool = True, skip_utf8_validation: bool = False, dispatcher: DispatcherBase | WrappedDispatcher | None = None, **_: Any)→ None [source]
```

Initialize WebSocket object.

**Parameters:** **sslopt** (*dict*) – Optional dict object for ssl socket options for details.

**abort()** [\[source\]](#)

Low-level asynchronous abort, wakes up other threads that are waiting in `recv_*`

**close(status: int = 1000, reason: str | bytes = b'', timeout: int | float | None = 3) → None** [\[source\]](#)

Close WebSocket object

- Parameters:**
- **status** (*int*) – Status code to send. See `VALID_CLOSE_STATUS` in ABNF.
  - **reason** (*str or bytes*) – The reason to close in UTF-8.
  - **timeout** (*int or float*) – Timeout until receive a close frame. If `None`, it will wait forever until receive a close frame.

**connect(url, \*\*options)** [\[source\]](#)

Connect to url. url is websocket url scheme. ie. `ws://host:port/resource` You can customize using 'options'. If you set "header" list object, you can set your own custom header.

```
>>> ws = WebSocket()
>>> ws.connect("ws://websockets.chilkat.io/wsChilkatEcho.ashx",
...           header=["User-Agent: MyProgram",
...                   "x-custom: header"])
```

- Parameters:**
- **header** (*list or dict*) – Custom http header list or dict.
  - **cookie** (*str*) – Cookie value.
  - **origin** (*str*) – Custom origin url.
  - **connection** (*str*) – Custom connection header value. Default value "Upgrade" set in `_handshake.py`
  - **suppress\_origin** (*bool*) – Suppress outputting origin header.
  - **suppress\_host** (*bool*) – Suppress outputting host header.
  - **host** (*str*) – Custom host header string.
  - **timeout** (*int or float*) – Socket timeout time. This value is an integer or float. If you set `None` for this value, it means "use default\_timeout value"
  - **http\_proxy\_host** (*str*) – HTTP proxy host name.
  - **http\_proxy\_port** (*str or int*) – HTTP proxy port. Required when `http_proxy_host` is set. Proxies from environment variables default to port 80.
  - **http\_no\_proxy** (*list*) – Whitelisted host names that don't use the proxy.
  - **http\_proxy\_auth** (*tuple*) – HTTP proxy auth information. Tuple of username and password. Default is `None`.
  - **http\_proxy\_timeout** (*int or float*) – HTTP proxy timeout. Default is 60 sec as per python-socks.
  - **redirect\_limit** (*int*) – Number of redirects to follow.



latest



- **subprotocols** (*list*) – List of available subprotocols. Default is None.
- **socket** (*socket*) – Pre-initialized stream socket.

**getheaders()** → dict | None [\[source\]](#)

Get handshake response header

**getstatus()** → int | None [\[source\]](#)

Get handshake status

**getsubprotocol()** → str | None [\[source\]](#)

Get subprotocol

**gettimeout()** → int | float | None [\[source\]](#)

Get the websocket timeout (in seconds) as an int or float

**Returns:**        **timeout** – returns timeout value (in seconds). This value could be either float/integer.

**Return type:**   int or float

*property* **headers:** dict | None

Get handshake response header

**ping(payload: str | bytes = "")** → None [\[source\]](#)

Send ping data.

**Parameters:**   **payload** (*str*) – data payload to send server.

**pong(payload: str | bytes = "")** → None [\[source\]](#)

Send pong data.

**Parameters:**   **payload** (*str*) – data payload to send server.

**recv()** → str | bytes [\[source\]](#)

Receive string data(byte array) from the server.

**Returns:**        **data**

**Return type:**   string (byte array) value.

**recv\_data(control\_frame: bool = False)** → tuple [\[source\]](#)

Receive data with operation code.

**Parameters:** **control\_frame** (*bool*) – a boolean flag indicating whether to return control frame data, defaults to False

**Returns:** **opcode, frame.data** – tuple of operation code and string(byte array) value.

**Return type:** tuple

```
recv_data_frame(control_frame: bool = False)→ tuple \[source\]
```

Receive data with operation code.

If a valid ping message is received, a pong response is sent.

**Parameters:** **control\_frame** (*bool*) – a boolean flag indicating whether to return control frame data, defaults to False

**Returns:** **frame.opcode, frame** – tuple of operation code and string(byte array) value.

**Return type:** tuple

```
recv_frame() \[source\]
```

Receive data as frame from server.

**Returns:** **self.frame\_buffer.recv\_frame()**

**Return type:** ABNF frame object

```
send(payload: bytes | str, opcode: int = 1)→ int \[source\]
```

Send the data as string.

**Parameters:**

- **payload** (*str*) – Payload must be utf-8 string or unicode, If the opcode is OPCODE\_TEXT. Otherwise, it must be string(byte array).
- **opcode** (*int*) – Operation code (opcode) to send.

```
send_binary(payload: bytes)→ int \[source\]
```

Send a binary message (OPCODE\_BINARY).

**Parameters:** **payload** (*bytes*) – payload of message to send.

```
send_bytes(data: bytes | bytearray)→ int \[source\]
```

Sends a sequence of bytes.

```
send_close(status: int = 1000, reason: str | bytes = b"")→ None \[source\]
```

Send close data to the server.

**Parameters:**

- **status** (*int*) – Status code to send. See STATUS\_
- **reason** (*str or bytes*) – The reason to close. This must be string or UTF-8 bytes.

**send\_frame(frame: *ABNF*)**→ int [\[source\]](#)

Send the data frame.

```
>>> ws = create_connection("ws://websockets.chilkat.io/wsChilkatEcho.ashx")
>>> frame = ABNF.create_frame("Hello", ABNF.OPCODE_TEXT)
>>> ws.send_frame(frame)
>>> cont_frame = ABNF.create_frame("My name is ", ABNF.OPCODE_CONT, 0)
>>> ws.send_frame(frame)
>>> cont_frame = ABNF.create_frame("Foo Bar", ABNF.OPCODE_CONT, 1)
>>> ws.send_frame(frame)
```

**Parameters:**    **frame** (*ABNF frame*) – frame data created by `ABNF.create_frame`

**send\_text(text\_data: str)**→ int [\[source\]](#)

Sends UTF-8 encoded text.

**set\_mask\_key(func)** [\[source\]](#)

Set function to create mask key. You can customize mask key generator. Mainly, this is for testing purpose.

**Parameters:**    **func** (*func*) – callable object. the func takes 1 argument as integer. The argument means length of mask key. This func must return string(byte array), which length is argument specified.

**settimeout(timeout: int | float | None)**→ None [\[source\]](#)

Set the timeout to the websocket.

**Parameters:**    **timeout** (*int or float*) – timeout time (in seconds). This value could be either float/integer.

**shutdown()** [\[source\]](#)

close socket, immediately.

**property status:** int | None

Get handshake status

**property subprotocol:** str | None

Get subprotocol

**property timeout:** int | float | None

Get the websocket timeout (in seconds) as an int or float

**Returns:**            **timeout** – returns timeout value (in seconds). This value could be either float/integer.

**Return type:**    int or float

```
websocket._core.create_connection(url: str, timeout: int | float | None = None, class_:
Type[WebSocket] = <class 'websocket._core.WebSocket'>, **options: Any)→ WebSocket \[source\]
```

Connect to url and return websocket object.

Connect to url and return the WebSocket object. Passing optional timeout parameter will set the timeout on the socket. If no timeout is supplied, the global default timeout setting returned by `getdefaulttimeout()` is used. You can customize using 'options'. If you set "header" list object, you can set your own custom header.

```
>>> conn = create_connection("ws://websockets.chilkat.io/wsChilkatEcho.ashx",
...     header=["User-Agent: MyProgram",
...             "x-custom: header"])
```

#### Parameters:

- **class** (*class*) – class to instantiate when creating the connection. It has to implement `settimeout` and `connect`. It's `__init__` should be compatible with `WebSocket.__init__`, i.e. accept all of it's kwargs.
- **header** (*list or dict*) – custom http header list or dict.
- **cookie** (*str*) – Cookie value.
- **origin** (*str*) – custom origin url.
- **suppress\_origin** (*bool*) – suppress outputting origin header.
- **host** (*str*) – custom host header string.
- **timeout** (*int or float*) – socket timeout time. This value could be either float/integer. If set to `None`, it uses the `default_timeout` value.
- **http\_proxy\_host** (*str*) – HTTP proxy host name.
- **http\_proxy\_port** (*str or int*) – HTTP proxy port. Required when `http_proxy_host` is set. Proxies from environment variables default to port 80.
- **http\_no\_proxy** (*list*) – Whitelisted host names that don't use the proxy.
- **http\_proxy\_auth** (*tuple*) – HTTP proxy auth information. tuple of username and password. Default is `None`.
- **http\_proxy\_timeout** (*int or float*) – HTTP proxy timeout, default is 60 sec as per python-socks.
- **enable\_multithread** (*bool*) – Enable lock for multithread.
- **redirect\_limit** (*int*) – Number of redirects to follow.
- **sockopt** (*tuple*) – Values for `socket.setsockopt`. `sockopt` must be a tuple and each element is an argument of `socket.setsockopt`.
- **sslopt** (*dict*) – Optional dict object for ssl socket options. See FAQ for details.
- **subprotocols** (*list*) – List of available subprotocols. Default is `None`.
- **skip\_utf8\_validation** (*bool*) – Skip utf8 validation.
- **socket** (*socket*) – Pre-initialized stream socket.



latest

